LIBXS Samples

Converse Sample

Converse reads plain text or Markdown files and then answers questions about them, summarizes them, assembles new text from them, and trains small next-token models on them. Answers are extracted from the ingested text and carry a citation, or the sample abstains; nothing is generated from outside the corpus unless you ask for it.

It is built only on libxs primitives (lexemes, metatokens, fingerprints, registries, n-grams, parameter prediction) and is self-contained: nothing is downloaded, no external service is contacted, and no data is used beyond the files you give it. It is experimental and adds no stable public API.

For the design and the measurements, see `~/papers/converse/paper.tex` and `~/papers/converse/insights.md`.

Build

Build the library first, then the sample:

```
cd ../..
make PEDANTIC=2
cd samples/converse
make PEDANTIC=2
```

Or, from the `libxs` root, `make PEDANTIC=2 samples/converse`.

Which binary

binary	serves	use it for
`converse-qa.x`	interactive, `-e`, `-c`, `-L`	questions, evaluation, recombination
`converse-lm.x`	`-E`, `-c -K KIND`, `-L`	next-token models, byte model
`converse.x`	everything above	anything, and all documented examples

`converse.x` accepts every command below. The split binaries link only what they serve, and a mode a binary does not serve is rejected up front, naming the one that does. One exception worth knowing: the byte model lives in the LM half, so under `converse-qa.x` the recombination probe prints `no seam judge` and leaves its bpc columns at zero. Use `converse.x` if you want those columns.

Run any binary without arguments to print its full option and environment list.

Getting text in

Put files anywhere; `texts/` is the conventional place and is ignored by git. Two structures are understood:

- **prose** — paragraphs of running text, blank-line separated. Uppercase lines are read as section headings and become the citation for everything under them.
- **Markdown** — headings, paragraphs and code blocks; `#` headings become the citation.

The structure is chosen by file extension (`.md` is Markdown, everything else is prose) and can be forced with `-p prose|markdown`.

Pass files directly, or use `-b PREFIX` to pick up a numbered set without listing it:

```
./converse.x texts/prose1.txt texts/prose2.txt
./converse.x -b texts/grimm                    # texts/grimm.txt, grimm-1.txt, ...
./converse.x -b docs ~/libxs/documentation/*.md
```

`-b PREFIX` probes `PREFIX`, `PREFIX.txt` and numbered parts (`PREFIX-N.txt`, `PREFIX_N.txt`, `PREFIX.N.txt`); missing siblings are fine as long as one file is usable. Markdown files are not probed, so pass them explicitly (a glob is fine, as above).

`-b` also names the **state files**: the basename of the prefix is the prefix of everything the run persists, so `-b docs` keeps `docs.dat`, `docs.lex` and so on, and several corpora can live side by side in one directory. Without `-b`, state is named `converse.*`.

Any UTF-8 text works. Suitable sources are Project Gutenberg books, a Wikipedia extract, your own notes, or the documentation of this repository. Larger corpora are supported; memory is the practical limit.

Warm and cold runs State is reused: a second run over the same files loads the persisted corpus, lexicon and reranker instead of re-ingesting, which is much faster. To force a full rebuild, delete the state files for that prefix:

```
rm -f grimm.dat grimm.par grimm.src grimm.lex grimm.prd grimm.facts
```

Do this before comparing two runs — a warm run answers from state built by the previous one.

To build the state without asking anything — useful before a batch of runs, or to time ingestion on its own — use `-L`:

```
./converse.x -L -b texts/grimm
```

Asking questions

```
./converse.x -b texts/grimm
```

The sample then reads one question per line:

```
> Who is Gretel?
Gretel is the girl.
citation: HANSEL AND GRETEL (texts/grimm.txt:2544)
> Where did Hans go?
Hans went into the stable.
citation: CLEVER HANS (texts/grimm.txt:6031)
> Who was eaten by the wolf?
Grandmother and Little Red-Cap were eaten by the wolf.
citation: LITTLE RED-CAP [LITTLE RED RIDING HOOD] (texts/grimm.txt:3104-3112)
> What belongs to Curdken?
hat belongs to Curdken.
citation: THE GOOSE-GIRL (texts/grimm.txt:1771)
> What do we know about Hansel?
Hansel is the boy. Hansel stood still and peeped back at the house.
citation: HANSEL AND GRETEL (texts/grimm.txt:2544-2579)
> Who is Sherlock Holmes?
I do not know from the corpus.
```

What is answerable depends on the rule files (next section). With the shipped rules the sample answers:

- `Who is X?` — an identity or a definition ("X is a Y", "X, a Y, . . . ").
- `Where did X go?` / `Where is X?` — a place, as a proposition from X's own sentence.
- `Who was V by X?` — a relation, including passives it was never told about.
- `What belongs to X?` — an enumeration, each item cited.
- `What do we know about X?` — several cited propositions collected about one name.
- `How are X and Y connected?` / `What connects X and Y?` — the path between two entities, stated as the propositions it is made of.
- anything else question-shaped — the best matching sentence, ranked and cited.

Every answer is followed by a citation naming where it came from: the section title when the corpus has headings, and always the file and the line. An answer assembled from several propositions cites the range of lines it rests on (`texts/grimm.txt:2544-2579`), one range per file, and a single line when it is one line. Titles are only printed when the text really carries them — a corpus of flat prose is cited by file and line alone rather than by a sentence that happened to look like a heading.

A connection question is answered by stating the facts that join the two entities, each with its own citation, and never by inventing a relation between them:

```
> How are McClellan and Douglas connected?
Lincoln restored McClellan. Lincoln forced Douglas.
citation: texts/wiki2m.txt:813-895
```

If nothing in the corpus joins them, the sample says so rather than answering with a sentence about one of them.

Three behaviours to expect:

- The sample **abstains** (`I do not know from the corpus.`) rather than guessing when the corpus does not support an answer.
- A question whose KIND it does not recognize — one using none of the declared `ask|` words — still gets an answer, introduced by `I did not recognize the question. The closest the corpus comes:` so a relevant sentence is never mistaken for a direct answer. That is different from abstaining: the question was not read, rather than read and unsupported.
- A question of the form `In TITLE, ...` is ranked only against the matching heading, which is how to ask about one document in a corpus of many.

Questions are case-insensitive.

Non-question input is treated as a composition request and answered from the fingerprint path instead. `-n N` sets how many sentences a response may use.

Checking answers against a fixture

`-e` runs a fixture of questions and expected terms and prints a pass count, which is how to tell whether a rule-file or corpus change helped:

```
./converse.x -e -b texts/grimm
```

The fixture is `<prefix>.eval` (so `grimm.eval` for `-b grimm`, `converse.eval` otherwise). Each non-comment line has three required fields and two optional ones:

```
question|evidence-terms|reply-terms|fact-terms|citation-terms
```

The fifth field checks the citation, and it is matched against the whole citation — title, file and line — so a fixture can assert `texts/grimm.txt:2544`.

Terms are comma-separated and matched case-insensitively. An empty evidence field marks an abstention case (the question SHOULD go unanswered); an empty reply field skips the concise-reply check. The fact field is checked only when relation facts were learned, so one fixture works with and without rule files.

`-P PROFILE` selects the profile used to rank candidate answers (`raw`, `poly2`, `smooth`, `temporal`, `rf`, `fisher`, `hknn`), for both interactive use and `-e`:

```
./converse.x -P temporal -e texts/prose1.txt
```

Summarizing and composing

`summarize.x` is a separate, smaller entry point:

```
./summarize.x texts/prose1.txt               # fuse adjacent sentences
./summarize.x -n 3 texts/prose1.txt          # to a target sentence count
printf 'First sentence. Second one.' | ./summarize.x -
./summarize.x -g -r -n 8 texts/prose1.txt texts/prose2.txt   # compose
```

- default: summarize one file by repeatedly fusing adjacent sentences.
- `-g`: compose mode — ingest all files, take the first as the target, and emit a short assembled text (`-n` is the phrase budget).
- `-r`: reflow text first, joining cosmetic line breaks (useful for Gutenberg files).

Recombining text from the corpus

An experimental mode splices clauses from different sentences into new ones, keeping each half verbatim and cutting only at clause boundaries:

```
CONVERSE_RECOMB=50 CONVERSE_HIER_RESCORE=1 ./converse.x -E -x -b texts/grimm
```

CONVERSE_RECOMB=N attempts N joins and reports how many were made, with coherence and seam statistics. CONVERSE_RECOMB_COMPOSE=1 additionally lets the interactive -c mode answer with a spliced sentence; it is off by default because every other mode emits text that occurs verbatim in the corpus and this one does not.

Next-token prediction

An experimental predictor is trained from the token stream and kept out of the question-answering path:

```
./converse.x -E -b texts/grimm                # accuracy, default trigram
./converse.x -E -K bigram -b texts/grimm      # pick the model
./converse.x -E -H 10 -b texts/grimm          # held-out: train on 9/10, test on 1/10
printf 'the little\n' | ./converse.x -c -b texts/grimm   # suggestions + continuation
CONVERSE_GRAN=meta-word ./converse.x -E -b texts/grimm   # token unit
```

- -E reports accuracy, -c prints the top few successors plus a short continuation.
- -H N evaluates on a held-out split, -T PREFIX on a separate corpus, which is what to use for an honest, non-memorized figure.
- -K selects the model: bigram, trigram (default), predict, embed, rerank, knnlm, or hier (evaluation-only: metatoken, byte and PPM bits per character).
- -P PROFILE selects the predictor profile for predict|embed|rerank (raw, poly2, smooth, temporal, rf, fisher, hknn).
- -x uses the deepest n-gram context.
- CONVERSE_GRAN sets the token unit: word, native, syllable, bpe, meta-native, meta-word, meta-syllable.

An optional <prefix>.predict fixture of context|expected-next lines adds a curated check to -E. Many further knobs exist; run the binary with no arguments to list them.

Teaching it a language and a corpus

Everything corpus- and language-specific lives in local rule files rather than in the source, and they are what decide which questions can be answered. Two files, both optional and both ignored by git:

- converse.rules — the LANGUAGE: function words and syntactic classes. A <prefix>.rules file REPLACES it (for a corpus in another language).
- <prefix>.relations — this CORPUS: aliases, role words, place names. It EXTENDS the language file, so it need not restate function words.

Each non-comment line is kind|term:

kind	declares	example
alias	a query verb and the verb the text uses	alias|eaten|devoured
person	a role word that can be bound to a name	person|grandmother
place	a place noun	place|forest
where, why, how	the markers those questions turn on	where|in
topic	the marker of an attribute question	topic|about
own	the verb a possession question uses	own|belongs
poss	how the language WRITES possession	poss|apostrophe-s
copula, article, prep	the syntactic classes definitions need	copula|is
aux	the auxiliaries	aux|had
agent	the word a passive names its agent with	agent|by
genitive	the word that marks a possessor after the thing	genitive|of
join	how the language writes a joiner inside a name	join|ampersand
link	the words that ask how two entities relate	link|connected
ask	a question KIND and this language's word for it	ask|who|who

kind	declares	example
`pron`	the back-reference pronouns a follow-up points with	`pron|it`
`result`	the light verb introducing a resulting state	`result|made`
`skip`, `negate`	filler words and negators	`skip|the`

A few notes that matter in use:

- `where|in` plus `place|forest` is what makes `Where ...?` answerable; declaring only one of the two answers nothing.
- `aux|had` and `agent|by` are enough to read passives and active clauses generally — the sample derives which words the corpus uses as verbs and as nouns from those frames and reports `verbs derived: N` and `nouns derived: N`. No verb list is needed.
- `poss|apostrophe-s` (English) against `poss|apostrophe` (German): declare the wrong one and possession answers go silent rather than wrong.
- `ask|` is how the sample knows a question's KIND. The first field is the kind (`who`, `what`, `where`, `when`, `why`, `how`, `yesno`), the second is the word — so a German rule file writes `ask|who|wer`. A question using none of the declared words still gets an answer, prefixed with a note that the question was not recognized.
- `person|father` plus `genitive|of` is what makes kinship answerable in BOTH forms — "Lincoln's father Thomas" and "Aegeus, the father of Theseus". A role word you do not declare is simply not read, so extend the `person|` class for the kinds of relation your text states.

The sample can also PROPOSE rules instead of only reading them: `CONVERSE_RULES_LEARN=N` offers up to N `person|` class members found in the corpus, which is a way to bootstrap a rule file for new text. What it proposes is speculative, so it is reported separately and, if a `<prefix>.learn.eval` fixture exists, that fixture is used for `-e` while learning is on.

With `CONVERSE_FACTS_LIST=1` the sample also prints the FACTS themselves, plus a `graph reach:` line (edges, entities, pairs joined by one middle, largest degree) — useful for judging whether a corpus is big enough to ask connection questions of. Prose of a few MB gives tens of edges; the tales give none, because a story names few entities that act on each other.

After ingestion the sample reports what it learned (`relation facts: N learned`, `identity facts`, `location facts`, `type facts`). Set `CONVERSE_FACTS_LIST=1` to print the facts themselves — reading them is the only way to tell whether they are true.

Optional `converse.bridges` adds evidence-backed answer frames, five fields per line:

```
name|query-groups|evidence-groups|score|reply
```

Whitespace separates required groups and `/` separates alternatives; `_` stands for a literal space in evidence terms. The reply is literal text or a small frame such as `{after:lighthouse had}`.

Files the sample writes

All state is kept in the working directory, named after the `-b` prefix (`converse.*` without one), and all of it is ignored by git:

file	holds
`<prefix>.dat`	the ingested corpus
`<prefix>.par`	the source texts the corpus refers to; required with `.dat`
`<prefix>.lex`	lexicon and token IDs
`<prefix>.prd`	the answer reranker
`<prefix>.src`	the file names the corpus refers to, for citations
`<prefix>.facts`	the derived fact layers, rebuilt when the corpus changes

Delete one of `.dat` / `.par` and both are rebuilt. The fixtures and rule files (`<prefix>.eval`, `<prefix>.learn.eval`, `<prefix>.predict`, `converse.rules`, `<prefix>.relations`, `converse.bridges`) are inputs you write; the sample never modifies them.

Foeppl Fingerprint

This sample generates paper-oriented experiments for the LIBXS Foeppl fingerprint API. It focuses on the fingerprint itself, leaving Rosetta as the higher-level algebra/type-discovery showcase.

What It Produces

Set `FPRINT_OUTDIR` to collect CSV artifacts:

```
mkdir -p results/fprint
FPRINT_OUTDIR=results/fprint ./fprint.x
```

The sample writes:

File	Purpose
`convergence.csv`	Same functions sampled at increasing resolution and compared to a high-resolution reference.
`sensitivity.csv`	Perturbations with matched element RMS but different structural character.
`geometry.csv`	Foeppl boundary area, centroid, moments, and shape fingerprints.
`hierarchy.csv`	2-D smooth, creased, diagonal-creased, and noisy fields in hierarchical and per-axis modes.
`compression.csv`	Newton truncation order versus reconstruction error.
`collision.csv`	Pseudometric collision test: pairs with shared L2 norms but different signed means.
`timing.csv`	Wall-clock cost per element at various input sizes.
`convergence_rate.csv`	Log-log convergence rate (slope and R-squared) for smooth functions.

Build

```
cd samples/fprint
make
```

Use From The Paper

The Foeppl paper driver `papers/foeppl/run.sh` builds this sample when needed and stores artifacts under `papers/foeppl/results/fprint` by default.

Batched GEMM

Exercises the LIBXS batched GEMM API (`libxs_gemm.h`) in several flavors: strided, pointer-array, indexed, and grouped. All programs are multithreaded via OpenMP, report wall-clock time and GFLOPS/s, and optionally validate results via `libxs_matdiff`.

Building

```
cd samples/gemm
make
```

LIBXS must be built first from the repository root.

Programs

gemm_strided Strided batch DGEMM: all matrices packed contiguously with constant stride. Uses `libxs_gemm_index` with index_stride=0.

```
./gemm_strided.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_batch Pointer-array batch DGEMM: matrices accessed through pointer arrays. Supports a duplicate C-matrix mode to exercise lock-forward sync.

```
./gemm_batch.x [M [N [K [batchsize [nrepeat [dup [beta [pad]]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, dup=0, beta=1.0, pad=0.

dup	Mode	Description
0	None	Each C-matrix is unique (LIBXS_GEMM_FLAG_NOLOCK)
1	Sorted	Half-unique C-pointers, duplicates consecutive
2	Shuffled	Half-unique C-pointers, randomly shuffled

gemm_index Index-array batch DGEMM: matrices in contiguous buffers, addressed through explicit element-offset index arrays (ia, ib, ic).

```
./gemm_index.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_indexf Fortran variant of gemm_index. Demonstrates the LIBXS Fortran module interface with one-based index arrays and C_LOC/C_SIZEOF interop. Requires a Fortran compiler.

```
./gemm_indexf.x [M [N [K [batchsize [nrepeat]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3.

gemm_groups Grouped batch DGEMM: multiple groups of different matrix shapes dispatched in sequence, each with its own `libxs_gemm_config_t`. Per-group dimensions grow by 4 starting from base_m.

```
./gemm_groups.x [ngroups [batch_per_group [nrepeat [base_m [beta [pad]]]]]]
```

Defaults: ngroups=2 (max 4), batch_per_group=30000, nrepeat=3, base_m=8, beta=1.0, pad=0.

Validation

Set the CHECK environment variable to validate results:

```
CHECK=1 ./gemm_strided.x
CHECK=1 ./gemm_batch.x 23 23 23 1000 1 2
CHECK=1 ./gemm_index.x
CHECK=1 ./gemm_groups.x
```

When padding is enabled (pad>0), the check also verifies that leading-dimension padding in C-matrices has not been overwritten.

Runtime Controls

Set `LIBXS_GEMM_PRINT=0` to print a compact GEMM registry summary at termination. The first line reports registry size, capacity, memory, and the selected `LIBXS_GEMM_BACKEND` policy; the second line reports a histogram by datatype and backend.

Memory Comparison Benchmark

Benchmarks `libxs_diff`, `libxs_memcmp`, and the C standard library's `memcmp` for both large contiguous comparisons and many small fixed-size comparisons. A Fortran variant compares `libxs_diff` against Fortran array intrinsics.

Building

```
cd samples/memory
make
```

Produces `memcmp.x` (C) and, if a Fortran compiler is found, `memcmpf.x` and `matcpyf.x`.

Usage (C)

```
./memcmp.x [elsize [stride [nelems [niters]]]]
```

Argument	Default	Description
elsize	INSIZE/MAXSIZE (compile, 64)	Element comparison size in bytes
stride	max(MAXSIZE, elsize)	Distance between successive elements
nelems	2 GB / stride	Number of elements
niters	5	Repetitions (arithmetic average)

Environment Variables

Variable	Default	Description
STRIDED	0	0: one-call when stride==elsize, else loop; 1: seq loop; >=2: random
CHECK	0	Non-zero: validation pass (inject diffs, cross-check implementations)
MISMATCH	0	0: equal buffers; 1: first byte; 2: middle; 3: last; >=4: random
OFFSET	0	Shift buffer b off its natural alignment by this many bytes

Compile-Time Knobs

Macro	Default	Description
MAXSIZE	64	Stride floor (elements spaced >= this far)
INSIZE	MAXSIZE	Default element size when argument is omitted

```
## 32-byte elements, sequential strided access, 10 repetitions
./memcmp.x 32 0 0 10

## 8-byte elements, random traversal, 3 repetitions
STRIDED=2 ./memcmp.x 8 0 0 3

## contiguous (single-call) comparison, ~2 GB
./memcmp.x 64 64 0 5

## mismatch at last byte, buffer b misaligned by 3 bytes
MISMATCH=3 OFFSET=3 ./memcmp.x 32 0 0 5
```

Environment variables and compile-time macros apply only to `memcmp.x`.

Usage (Fortran — memcmpf)

```
./memcmpf.x [nelements [nrepeat]]
```

Element type is hard-coded as INTEGER(4). Compares ~2 GB by default. Reports per-iteration times (ms) and average throughput (MB/s).

Usage (Fortran — matcpyf)

```
./matcpyf.x [m [n [ldi [ldo [nrepeat [nmb]]]]]]
```

Argument	Default	Description
m	4096	First matrix dimension
n	m	Second matrix dimension
ldi	m (enforced m)	Leading dimension of input
ldo	ldi	Leading dimension of output
nrepeat	2	Number of repetitions
nmb	2048	Memory budget in MB

Benchmarks `libxs_matcopy` and `libxs_matcopy_task` for matrix copy and zeroing. When the matrix is square and ldi==ldo, also benchmarks `libxs_otrans` and `libxs_otrans_task` for out-of-place transpose.

What Is Measured

Three comparison implementations are benchmarked:

- `libxs_diff` — SIMD-dispatched short-buffer comparison (SSE/AVX2/AVX-512). Limited to elsize < 256 bytes, per-element.
- `libxs_memcmp` — SIMD-dispatched memcmp replacement. Single call for contiguous buffers, or per-element for strided access.
- stdlib `memcmp` — C library implementation under the same pattern.

Each kernel gets freshly initialized data before its timed section. An untimed warm-up pass runs before the first measured iteration. The summary reports min / median / max across iterations plus peak MB/s. Reported MB/s counts both buffers (2 * nbytes / time).

Ozaki-Scheme Low-Precision GEMM

Intercepts DGEMM, SGEMM, ZGEMM, and CGEMM at link time, replacing them with low-precision Ozaki schemes (mantissa slicing or CRT). Real GEMM calls go through the selected scheme; complex GEMM (ZGEMM/CGEMM) is mapped to real GEMM internally.

Build

```
cd samples/ozaki
make -j $(nproc)
```

LIBXS must be built first from the repository root. When a sibling `libxstream` directory is detected, the optional OpenCL/GPU path is compiled in (see OZAKI_OCL below).

Link-Time Interception

Two link-time variants are built per precision:

- `dgemm-blas.x` / `sgemm-blas.x` — dynamically linked against BLAS
- `dgemm-wrap.x` / `sgemm-wrap.x` — linked with `--wrap=` (GNU ld)
- `zgemm-wrap.x` / `cgemm-wrap.x` — complex GEMM wrappers

Static (`libwrap.a`) and shared (`libwrap.so`) wrapper libraries are built by default. The shared library can intercept any application:

```
LD_PRELOAD=/path/to/libwrap.so ./application
```

The static library requires explicit `--wrap` flags:

```
-Wl,--wrap=dgemm_ -Wl,--wrap=sgemm_ -Wl,--wrap=zgemm_ -Wl,--wrap=cgemm_
```

Test scripts: `test-wrap.sh` exercises all variants, `test-check.sh` validates both schemes, `test-mhd.sh` tests MHD file-based input.

Test Driver

```
dgemm-wrap.x [A.mhd|M [B.mhd|N [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]]
```

Defaults: M=N=K=257, TA=TB=0, ALPHA=BETA=1.0. The first argument can be 0 followed by MHD filenames to load A and B matrices from files. The `zgemm-wrap.x` and `cgemm-wrap.x` drivers call ZGEMM and CGEMM.

Environment Variables

Scheme Selection

Variable	Default	Description
OZAKI	2 (CPU)	0=bypass (BLAS), 1=mantissa slicing, 2=CRT, 3=adaptive
OZAKI_COMPLEX	(auto)	Complex dispatch: 0=BLAS, 1=CPU, 2=GPU+fallback. Auto: 2 if on
OZAKI_N	(auto)	Slices (Sch.1: fp64=8, fp32=4) or primes (Sch.2: fp64=16, fp32=9)

OZAKI=3 (adaptive) starts with Scheme 1 on the first GPU call to learn the effective cutoff from preprocessing occupancy. Subsequent calls compare the Scheme-1 pair count (at the cached cutoff) against the Scheme-2 prime count and pick the cheaper path. On CPU, adaptive falls back to Scheme 2.

Unset means CRT on the CPU and adaptive on the GPU: the GPU default is LIBXSTREAM's, so that it stays the same whichever driver asks for it. Setting OZAKI applies to both.

Accuracy

Variable	Default	Description
OZAKI_FLAGS	3	Sch.1 bitmask: 1=Triangular, 2=Symmetrize, 0=full S^2 square
OZAKI_TRIM	0	Levels to trim (0=default). ~7 bits/level (Sch.1), ~4 bits (Sch.2); negative buys prec
OZAKI_I8	0	Sch.2: signed i8 residues (moduli<=128) instead of u8
OZAKI_GROUPS	0	Sch.2: K-grouping (0/1=off). Consecutive K panels, one reconstr.
OZAKI_MAXK	32768	Max K per preprocessing pass (0=full K in one pass)
OZAKI_THRESHOLD	12	Intensity threshold. Bypass when flops/(bytes*thr)<1. 0=always

GPU Path (requires LIBXSTREAM)

Variable	Default	Description
OZAKI_OCL	0	Enable OpenCL/GPU path (0=off, 1=on)
OZAKI_TM	(auto)	GPU output tile height (multiple of 8)
OZAKI_TN	(auto)	GPU output tile width (multiple of 16)

GPU-specific kernel tuning variables (OZAKI_RTM, OZAKI_RTN, OZAKI_WG, OZAKI_SG, OZAKI_KU, OZAKI_RC, OZAKI_PB, OZAKI_HIER, OZAKI_PREFETCH, OZAKI_SCALAR_ACC, OZAKI_CACHE, OZAKI_ARENA) are documented in the LIBXSTREAM Ozaki README.

Monitoring and Diagnostics

Variable	Default	Description
OZAKI_VERBOSE	0	0=silent, 1=stats at exit, N=print every Nth GEMM call
OZAKI_STAT	0	Track C-matrix (0), A-representation (1), or B-representation (2)
OZAKI_DUMP	0	Dump A/B as MHD files at the given call-count (0=off)
OZAKI_EPS	inf	Dump A/B when epsilon error exceeds threshold
OZAKI_RSQ	0	Dump A/B when RSQ drops below threshold (updated after dump)
OZAKI_EXIT	1	Exit on accuracy violation after dump. 0=continue

Benchmark

Variable	Default	Description
CHECK	0	Validate vs BLAS: 0=off, negative=auto-threshold, positive=custom
GRADE	0	Componentwise accuracy grade vs BLAS (see below)
NREPEAT	3	Number of GEMM calls (first call is warmup when >1)
EVIL	0	Adversarial exponent-span test (see below)
TAME	0	Keep n mantissa bits and clear the rest (see below)
OZAKI_DECAY	0	Decay diagnostic + K-permutation (0-4, see below)
GEMM_HOSTMEM	0	Operands from LIBXSTREAM (page-locked) instead of `malloc`

`GEMM_HOSTMEM` selects how the operands are allocated, which is what an application would do differently rather than a property of the scheme: the default is plain `malloc`, declared to LIBXSTREAM so that transfers can still take the faster route, whereas a non-zero value allocates page-locked memory through LIBXSTREAM. On a PCIe part the difference is worth several times the transfer time, so a comparison against another library has to state which of the two it used. The default keeps a CPU-only run off the GPU entirely: no device is opened unless the GPU path is enabled (`OZAKI_OCL=1`), while `GEMM_HOSTMEM` needs a device to allocate from and therefore brings one up.

Adversarial Input (EVIL) The EVIL variable initializes A and B with controlled exponent structure for stress-testing accuracy and adaptive slice reduction. The magnitude sets the exponent span in bits; the sign selects the distribution:

EVIL=N (N>0) Per-column. Column j of A is scaled by 2^(-N + round(2*N*j/(ncols-1))), column j of B by the inverse, so the span is 2N binades centred on zero. Product A*B is well-conditioned. Uniform exponents within each column.

EVIL=-N (N>0) Per-element. Each element gets a pseudorandom exponent in [0,N] via coprime shuffle, with opposite sign for B. Every row of A spans the full exponent range — worst case for row-wise alignment and adaptive cutoff.

EVIL=0 Default shuffle mode (no exponent structure).

The per-column mode (EVIL>0) matches the NVIDIA emulation grading test (diagonal scaling with D and D^-1), where EVIL is that test's exponent-range parameter b. The per-element mode (EVIL<0) is adversarial for the adaptive slice-pair reduction: it forces all slices to be populated in every row.

TAME is the sibling knob on the significand axis: `TAME=n` keeps n mantissa bits and clears the rest, so `TAME=24` is data promoted from single precision. The two axes are independent, and the generator carries a version (`LIBXS_MATRNG_VERSION`, reported per run as `DATA: matrng=...`) because a change to it makes results incomparable in a way nothing downstream can see.

Accuracy Grade (GRADE)

`GRADE=1` applies the componentwise criterion of the graded BLAS accuracy tests: `|C - C_ref| <= f(K) * u * (|alpha||A||B| + |beta||C|)` with `f(K) = K` and `u` the unit roundoff. K is the number of terms summed, so a short-K GEMM is held to a tighter bound than a square one of the same size. It reports `GRADE: a=<max ratio> f(n)=<K>` and fails the run when the ratio exceeds it. `GRADE=-1` reports without failing, which is what a deliberately trimmed run needs: it gives up precision on purpose, so this is not the criterion it claims.

Prefer it over `CHECK` when the question is whether a result is as accurate as the scheme claims. `CHECK` compares one scalar against a fixed threshold (1e-10 for double, 1e-3 for single), which is orders looser than these schemes deliver and therefore admits results that are lossy but not broken. `rsq` substitutes for neither: it saturates at 1 unless the output degenerates. GRADE costs one extra reference GEMM and one m-by-n buffer.

Decay Diagnostic and K-Permutation (OZAKI_DECAY)

Controls K-dimension row reordering for smoothness and the forward-difference decay diagnostic. Values map to libxs_sort_t:

Value	Behavior
0	Off (default). No permutation, no diagnostic.
1	Identity permutation. Measure decay on original ordering.
2	Sort B rows by L1-norm, apply same K-permutation to A.
3	Sort B rows by mean value, apply same K-permutation to A.
4	Greedy nearest-neighbor chain on B rows (O(K^2*N)).

Values 2-4 compute a K-permutation from B (via libxs_sort_smooth) before Ozaki-1 slicing. Both A and B are sliced using the permuted K-order. Since C = A*B = (A*P^T)*(P*B) for any permutation matrix P, the result is mathematically identical — only the int8 digit structure along K changes.

When nonzero, reports the forward-difference decay of int8 slice buffers along K, M, and N axes (first K-group only, single- threaded). Uses libxs_fprint per-axis mode internally. Output goes to stderr:

OZ1[MxNxK] Delta-K: d1=... d2=... ... OZ1[MxNxK] Delta-M: d1=... d2=... ... OZ1[MxNxK] Delta-N: d1=... d2=... ...

Decaying values indicate exploitable smoothness; growing values (~2x per order) indicate unstructured data where data-independent schemes (Ozaki-1, Ozaki-2) are well-matched.

Profiling

Set `LIBXSTREAM_PROFILE=1` for per-kernel device timings, reported at program exit:

```
PROF ACC/OpenCL: ID=171071 gemm_crt_fused=2.987 ms 5751.0 GFLOPS/s
```

There is no separate host-side profile: on the CPU the intercepted call and the kernel it spends its time in agree to within a few percent, so the `OZAKI GEMM` line already reports the kernel. It is a mean over the whole timed loop, so use `NREPEAT` to average several calls — host timings vary between runs, the more so on a many-core part.

Example

```
./dgemm-wrap.x 256                              # CRT scheme (default)
OZAKI=1 ./dgemm-wrap.x 256                      # mantissa slicing
OZAKI_TRIM=2 OZAKI=1 ./dgemm-wrap.x 256         # drop 2 diagonals below the default
OZAKI=2 OZAKI_GROUPS=4 ./dgemm-wrap.x 4096      # CRT with K-grouping
OZAKI=3 ./dgemm-wrap.x 4096                     # adaptive scheme selection
EVIL=512 ./dgemm-wrap.x 1024                    # accuracy grading (wide exponent span)
EVIL=1 ./dgemm-wrap.x 1024                      # narrow span (cutoff drops pairs)
EVIL=-52 ./dgemm-wrap.x 1024                    # per-element span (worst for cutoff)
```

```
OZAKI_DECAY=1 OZAKI=1 ./dgemm-wrap.x 256    # decay diagnostic (no reorder)
OZAKI_DECAY=2 OZAKI=1 ./dgemm-wrap.x 256    # sort by norm + decay diagnostic
OZAKI_DECAY=4 OZAKI=1 ./dgemm-wrap.x 256    # greedy NN + decay diagnostic
```

Predict Samples

Nine executables demonstrating fingerprint-guided prediction:

- **predict_params** — Parameter prediction from structured CSV (GPU kernel tuning, configuration databases).
- **predict_sunspots** — Timeseries forecasting via sliding-window kNN (monthly sunspot numbers, 1749-present).
- **predict_earthquakes** — Spatial prediction of earthquake magnitude from location and depth (USGS catalog).
- **predict_discharge** — River discharge forecasting via sliding-window kNN with day-of-year seasonality (USGS NWIS daily streamflow).
- **predict_soi** — Southern Oscillation Index prediction from anti-correlated Tahiti/Darwin sea level pressure using SPREAD decomposition (sum/diff modes).
- **predict_stock** — Paired-stock timeseries prediction from CSV using SPREAD decomposition on two correlated price series.
- **predict_crystal** — Crystal system classification from composition features (AFLOW ICSD, 7 classes, 60K entries).
- **predict_bits** — Held-out code length under the model against the training distribution: whether the inputs inform the output at all.
- **predict_ett** — ETT (Electricity Transformer Temperature) hourly forecasting with univariate, multivariate, PCA, and local-attention modes (ETTh1, standard benchmark for timeseries LLMs).

Build

```
make
```

Or from the LIBXS root:

```
make samples/predict
```

XGBoost Comparison

`predict_params`, `predict_crystal`, `predict_earthquakes` and `predict_ett` accept an `xgb` keyword that trains XGBoost on **exactly** the entries LIBXS was built from and reports both models side by side. It is compiled in when XGBoost is found, which needs the development headers (a pip-installed `xgboost` ships the shared library only):

```
XGBOOST_ROOT=/path/to/xgboost make    # explicit location
make                                  # or found via pkg-config
XGB=0 make                            # never compile it in
```

Requesting `xgb` from a binary built without it is an error, not a silent LIBXS-only run.

Keywords may be written anywhere on the command line and numbers fill the positional slots in order, so `predict_crystal.x data.csv xgb` and `predict_crystal.x data.csv 0.8 2 0 xgb` are the same run. A token that is neither a number nor a known keyword is refused: it used to land in the next numeric slot as `atof("xgb") == 0`, which trained on two rows and reported an accuracy for them. `predict_params` is the exception in that its keywords must still precede the file names, which cannot be told from keywords by shape.

Per-output, XGBoost is posed the same task LIBXS chose for itself: a classification over the attested value set where LIBXS classifies, a regression where it interpolates. An output with a single attested value is reported as constant and not trained (five of the sixteen kernel parameters are constant in `tune_multiply_PVC.csv`, so an average over all outputs is dominated by columns both models get for free).

Hyperparameters are fixed so the numbers are reproducible, and overridable:

Variable	Default	Meaning
`XGB_ROUNDS`	200	Boosting rounds
`XGB_DEPTH`	6	`max_depth`

Variable	Default	Meaning
`XGB_ETA`	0.1	Learning rate
`XGB_NTHREAD`	0	Threads (0 = XGBoost default)
`XGB_REGOBJ`	*(per sample)*	Regression objective
`XGB_LATENCY`	4096	Rows the per-query timing uses (0 = skip)

Where a sample reports timings it reports XGBoost's phases apart - marshalling the corpus into XGBoost's layout, the boosting rounds, and the batched prediction

- because only the rounds are the counterpart of a LIBXS build, and a single figure covering all three had been read as if it were. The per-query line is the same distinction for inference, and it is reported on both sides: one query at a time (`libxs_predict_eval`, and `XGB_LATENCY` rows through XGBoost's inplace prediction) and a whole matrix at once (`libxs_predict_eval_batch_task` across the threads, and XGBoost's batched call). Comparing one library's batch against the other's per-query loop measures the two interfaces, not the two models.

`XGB_REGOBJ` matters where the reported metric is mean absolute error: `predict_earthquakes` therefore proposes `reg:absoluteerror`, and forcing `reg:squarederror` there moves the magnitude error 0.237 to 0.254, which is enough to invert the comparison. `predict_params` and `predict_crystal` propose `reg:squarederror`.

`predict_crystal` additionally reads `GATE`: a comma-separated list of confidence thresholds (default `0.9`). The first entry drives the confidence-gated line; supplying more than one also prints a precision/coverage sweep for both models. Comparing two confidence signals at one threshold compares two different operating points, so a sweep is the only way to tell a better-calibrated signal from a differently-scaled one:

```
GATE=0.5,0.6,0.7,0.8,0.9,0.95,0.99 ./predict_crystal.x predict_crystal.csv 0.8 2 0 xgb
```

```
./predict_params.x 0.8 mix xgb ../smm/params/tune_multiply_PVC.csv
./predict_crystal.x predict_crystal.csv 0.8 2 0 xgb
```

The `mix` keyword matters for the parameter CSVs: they are sorted by problem size, so the default prefix split holds out the *largest* shapes and measures extrapolation, which changes which model looks better. Use `mix` for interpolation, omit it for extrapolation, and say which one a number came from.

Exact-match on kernel parameters is a proxy for kernel quality: the shipped CSVs carry no `GFLOPS`, so a differently-parameterized kernel that performs identically counts as a miss for both models.

predict_params

Train a prediction model from a CSV file and save it for later use. Reports validation quality on a held-out subset.

```
./predict_params.x [fraction] [auto|cat|compress[Q]|interp|rf|hknn|xgb] [-N] <csvfile> [modelfile [con

fraction   Validation split 0..1 for quality report (default: 0.8).
auto       Auto-detect mode per output (default).
cat        Force categorical (kNN) for all outputs.
compress   Drop redundant entries (Q: threshold, default 0.9).
interp     Force interpolation for all outputs.
rf         Random Forest classification.
hknn       Hierarchical kNN (Fisher-guided partition).
xgb        Also train XGBoost on the same split and compare.
-N         Max polynomial order (default: 0 = auto).
csvfile    Delimited text file.
modelfile  Output path for the binary model.
confidence-prefix  Optional prefix for confidence map MHD files.
```

Naming a mode fixes it: the saved model keeps only what that mode reads and cannot be evaluated by another one. Under `auto` the model stays open to all of them. The difference shows on `rf`, which answers from its trees and therefore stores no partition.

```
./predict_params.x ../../samples/smm/params/tune_multiply_PVC.csv
./predict_params.x 0.8 hknn tune_multiply_PVC.csv model.bin
```

predict_sunspots

Timeseries forecasting using sliding-window nearest-neighbor prediction.

```
./predict_sunspots.x <csvfile> [train_fraction] [compress[Q]] [hknn|rf]

NOPHASE=1  Drop the solar-cycle phase input.
NOBANK=1   Use a single window view instead of the bank.
WINDOW=n   Fix the window (default: auto).  WINDOW=-1 selects it by
           trial instead, which costs about 7x the build time and picks
           9 rather than 14 here (mean error over six months 18.71 ->
           18.30).  WINDOW=-w also offers w as a candidate.
```

```
./predict_sunspots.x predict_sunspots.csv 0.8
```

The sample carries one auxiliary input alongside the window (libxs_predict_set_series_aux): the phase of the solar cycle, measured from the training slice so nothing is tuned by hand. Longer horizons gain the most (t+6 23.70 -> 21.77, t+3 20.72 -> 18.94); t+1 is unchanged. NOPHASE=1 drops it.

The forecast is averaged over two views of the window (libxs_predict_set_series_bank), the second reading half the lags of the first: t+1 improves 17.52 -> 16.79 and t+6 20.30, at no extra corpus. Pass a larger count for more views (a third reaches 20.14 at t+6); NOBANK=1 drops back to one. Views are unavailable under a decomposition, where the inputs are no longer lags.

Data Source Monthly mean total sunspot number from SILSO (World Data Center, Royal Observatory of Belgium). Semicolon-delimited: year, month, decimal_year, sunspot_number.

predict_earthquakes

Predict earthquake magnitude from geographic location and depth.

```
./predict_earthquakes.x <usgs_csv> [train_fraction] [compress[Q]] [hknn|rf] [xgb]

xgb        Also train XGBoost on the same split and compare.
```

The kNN vote reports the median rather than the mean here. libxs_predict_set_central picks that per output at build time and needs no argument (0.265 -> 0.249 for kNN, 0.263 -> 0.241 for hknn); pass 1 or 2 to force median or mean.

```
./predict_earthquakes.x predict_earthquakes.csv
```

Data Source USGS Earthquake Hazards Program (public domain). Comma-delimited: time, latitude, longitude, depth, mag, ...

predict_discharge

River discharge forecasting with day-of-year seasonality and log-transform on outputs for heavy-tailed data.

```
./predict_discharge.x <discharge_tsv> [train_fraction] [compress[Q]] [hknn|rf]
```

```
./predict_discharge.x predict_discharge.tsv
```

Data Source USGS National Water Information System (public domain). Colorado River at Lees Ferry, site 09380000. Tab-delimited RDB format.

predict_soi

Southern Oscillation Index prediction from anti-correlated sea level pressure at Tahiti and Darwin using SPREAD decomposition.

```
./predict_soi.x <tahiti_file> <darwin_file> [train_fraction] [compress[Q]] [hknn|rf]
```

```
./predict_soi.x predict_soi_tahiti.dat predict_soi_darwin.dat
```

Data Source NOAA Climate Prediction Center (public domain). Fixed-width monthly sea level pressure (mb above 1000 mb).

predict_stock

Multi-stock timeseries prediction with auto-differencing and PCA/SPREAD decomposition.

```
./predict_stock.x <csv_file> [columns] [train_fraction] [compress[Q]] [hknn|rf]

columns    Comma-separated 0-based column indices (default: 1,2).
```

```
./predict_stock.x stocks.csv 1,2,3
```

predict_crystal

Crystal system prediction (7-class classification) from chemical composition features.

```
./predict_crystal.x <crystal_csv> [train_fraction] [order] [nclusters] [compress[Q]] [fisher|hknn|setd

fisher      Fisher discriminant feature weighting.
hknn        Hierarchical kNN (Gini-guided partition).
setdiff     Setdiff feature selection.
rf          Random Forest classification (default).
none        Raw kNN without feature processing.
xgb         Also train XGBoost on the same split and compare.
```

```
./predict_crystal.x predict_crystal.csv
```

Data Source AFLOW ICSD catalog (free for academic use). 60,386 entries with Magpie-style composition features (37 features). Crystal systems: triclinic(1), monoclinic(2), orthorhombic(3), tetragonal(4), trigonal(5), hexagonal(6), cubic(7).

predict_bits

Code length of a held-out stream in bits per event, under the model and under the training distribution of the same output. Use it to ask whether the inputs inform the output at all, which an error figure does not answer.

```
./predict_bits.x <csvfile> <innames> <outname> [fraction] [vocabulary] [hknn]

vocabulary  Distinct values the caller considers possible.  0 uses the
            attested support plus one aggregate novel atom, so an
            unattested outcome costs infinite bits; pass a count above
            the support size to keep the figure finite.
```

Example Run a control alongside any such measurement: a null result is also what a broken setup reports, so pair the question with one whose answer is known.

```
./predict_bits.x predict_earthquakes.csv latitude,longitude,depth mag 0.8 68
./predict_bits.x predict_earthquakes.csv latitude,longitude,mag depth 0.8 9000
```

Magnitude is not determined by location (3.265 bits against the training distribution's 3.248) and depth is (1.478 against 7.615).

predict_ett

ETT (Electricity Transformer Temperature) hourly forecasting. Supports univariate and multivariate modes with PCA decomposition and per-query local-correlation attention. Standard benchmark for comparison against transformer-based timeseries models.

```
./predict_ett.x <ett_csv> [nseries=1..7] [attend|spread|pca|hknn|rf|nocompress|xgb]

nseries     Number of input channels (1=OT only, 7=all).
attend      Per-query local-correlation channel weighting.
spread      Sum/diff decomposition across channels.
pca         PCA rotation of multi-channel input space.
xgb         Also train XGBoost on the same windows and compare.
```

`xgb` reads the built sliding windows back out of the model, so it implies `nocompress` (compression prunes entries before they can be read) and is refused alongside `attend`, a decomposition, or `BANK`, each of which zeroes the weight of lags the distance does not read. The effective window applies to both models, so `WINDOW=96` compares them at the conventional input length rather than at the auto-selected one — the two rank differently at the two lengths.

```
./predict_ett.x predict_ett.csv              # univariate OT
./predict_ett.x predict_ett.csv 2 pca        # 2ch with PCA (best MSE)
./predict_ett.x predict_ett.csv 7 attend     # 7ch with local-attention
./predict_ett.x predict_ett.csv 7            # 7ch raw (baseline)
```

Data Source ETTh1 (Electricity Transformer Temperature, hourly) from Zhou et al. 2021 (Informer). 17,420 hourly readings of an oil-filled electrical transformer (July 2016 - June 2018). Seven channels: HUFL, HULL, MUFL, MULL, LUFL, LULL, OT. Target: OT (oil temperature). Standard split: train months 1-12, test months 17-24, horizon H=96 steps (4 days).

Download: github.com/zhouhaoyi/ETDataset/blob/main/ETT-small/ETTh1.csv Rename to predict_ett.csv (CRLF line endings accepted).

Registry Microbenchmark

Benchmarks the performance of the LIBXS registry (key-value store) dispatch path under different access patterns and concurrency levels.

Programs

```
./registry.x [total [nrepeat [nthreads]]]
```

Argument	Default	Description
total	10000	Number of unique keys to register (min 2)
nrepeat	10	Repeat iterations for lookup phases
nthreads	max available	Number of OpenMP threads

Measurements:

- Duration to register (insert) all keys into the registry.
- Cold lookup with shuffled access pattern (defeats thread-local cache).
- Cached lookup with sequential/repeating pattern (hits thread-local cache).
- Multi-threaded parallel reads across all threads.
- Contended parallel writes (each thread registers its own key range).
- Mixed read/write: one writer thread while remaining threads read.

The multi-threaded benchmarks require at least 2 threads and are skipped when running single-threaded.

```
./registryf.x
```

Fortran variant with hardcoded parameters (10000 keys, 10 repeats). Measures registration, cold lookup, and cached lookup.

Scaling Behavior

Read-only accesses stay roughly constant in per-op duration due to the thread-local cache. Write accesses are serialized and duration scales with the number of threads.

Rosetta

Demonstrates hierarchical type discovery on opaque binary data. A table of harmonic-series values is encoded as a flat byte blob with no metadata. The analysis rediscovers the element type, record stride, and per-field structure — recovering mathematical content from anonymous bytes.

What It Shows

The sample proceeds through the levels described in the algebra paper (Section "Hierarchical Type Discovery"):

Step	Tool used	What it finds
Flat probe	libxs_fprint (UNKNOWN)	inconclusive (expected)
Stride sweep	libxs_fprint per-column	f64, stride=6
Shuffle test	libxs_shuffle + fprint	order matters (3-4x)
Field analysis	libxs_fprint per-field	decay rates per column
Sort test	libxs_sort_smooth (GREEDY)	already optimally ordered
Verification	libxs_setdiff_min	0 unmatched, tol=0

Key observations from the output:

- The flat 1-D probe fails because interleaved fields of different scales (1/n mixed with n mixed with ln(n)) look like noise when read sequentially. This motivates the stride sweep.
- The stride sweep requires ALL columns at a candidate width to have decay < 1 before accepting it, which eliminates false positives from partial correlations at wrong strides.
- Shuffle stability confirms that the record order carries genuine structure (decay increases ~4x after coprime permutation).
- Per-field analysis reveals:
  - `[0]` decay=0 — perfect ramp (sequential index)
  - `[1]` decay~0.63 — 1/n (decaying but not ultra-smooth)
  - `[2]` decay~0.20 — H_n (partial sums, very smooth)
  - `[5]` decay~0.29 — converges to Euler-Mascheroni gamma
- GREEDY sort confirms the data is already optimally ordered (the natural 1..64 sequence is the smoothest permutation).

The Data

For n = 1, 2, . . . , 64 the table stores six f64 fields per record:

```
[0] n             integer index
[1] 1/n           harmonic term
[2] H_n           partial sum of the harmonic series
[3] ln(n)         natural logarithm
[4] H_n - ln(n)  difference (converges to gamma from above)
[5] H_n - ln(n) - 1/(2n)   better gamma estimate
```

Total: 64 records x 6 fields = 384 doubles = 3072 bytes.

Build

```
cd samples/rosetta
make
```

Usage

```
./rosetta.x
```

No arguments. The sample is self-contained: it generates the data, encodes it as a flat blob, runs the full analysis, and prints the results.

To collect machine-readable artifacts, set `ROSETTA_OUTDIR` to an existing directory:

```
mkdir -p results/rosetta
ROSETTA_OUTDIR=results/rosetta ./rosetta.x
```

This writes `summary.csv`, `rosetta_original.mhd`, `rosetta_shuffled.mhd`, `rosetta_fields.mhd`, and `rosetta_fields_shuffled.mhd`. If `LIBXS_MHD_PNG=1` is also set, the MHD writer emits PNG files with the same basenames. The `fields` images are normalized per field, making the ordered and shuffled record traces easy to compare visually. The artifact mode is optional and leaves the interactive sample output unchanged.

Example Output

```
ROSETTA: Recovering structure from opaque bytes
Ground truth (hidden from the analysis):
  64 records x 6 fields = 384 doubles (3072 bytes)
  Data: harmonic series H_n and derived quantities.
  Field [5] converges to Euler-Mascheroni gamma.
Level 0: Flat 1-D probe (all bytes as one sequence)
  Input: 3072 opaque bytes
  No type has decay < 1 -- flat stream is not smooth.
  This is expected: interleaved fields of different
  scales look like noise when read sequentially.
Stride sweep: discovering record layout
  Best type:   f64
  Best stride: 6 elements (48 bytes per record)
  Records:     64
  Avg decay:   0.288244
Shuffle stability (record-level)
  Avg decay (original): 0.288244
  Avg decay (shuffled): 1.103839
  Ratio:                3.8x
  -> Record order carries structure.
Per-field decay analysis
  [0] n          decay=0.000000
  [1] 1/n        decay=0.631512
  [2] H_n        decay=0.203226
```

```
  [3] ln(n)       decay=0.259621
  [4] H_n-ln(n)   decay=0.342853
  [5] gamma_est   decay=0.292251  (last=0.5771953203, gamma=0.5772156649)
  Smoothest: [0] n (perfect ramp, decay=0)
  -> This reveals a sequential index column.
  Most interesting: [5] gamma_est -- converges to a constant.
GREEDY sort test (64 rows x 6 cols)
  Data is already optimally ordered for row smoothness.
Verification: setdiff(original, blob)
  Unmatched: 0, tolerance: 0.00e+00
  -> Byte-perfect recovery.
```

Why This Is Interesting

From 3072 anonymous bytes the framework discovers:

1. The element type (f64) — not i32, not f32, not i64.
2. The record structure (6-field records of 48 bytes each).
3. A sequential index column (field [0], decay = 0).
4. A column converging to Euler-Mascheroni gamma (field [5]).
5. That the natural ordering is already optimal (no resorting).

No metadata, no format knowledge, no human guidance. The hierarchical composition — stride sweep with all-columns-must-pass filtering, shuffle stability, per-field decay ranking — is what makes this possible. Any single tool alone would either fail (flat probe) or produce ambiguous results (stride sweep without the all-columns requirement accepts wrong strides).

Memory Allocation (Microbenchmark)

Benchmarks pooled scratch memory allocation (`libxs_malloc` / `libxs_free`) against the standard C library `malloc` / `free`. The pool-based allocator recycles memory after an initial warm-up, avoiding repeated system calls in steady state. A Fortran variant compares `libxs_malloc` against Fortran ALLOCATE / DEALLOCATE.

Building

```
cd samples/scratch
make
```

Produces `scratch.x` (C) and, if a Fortran compiler is found, `scratchf.x`.

Usage (C)

```
./scratch.x [ncycles [max_nactive [nthreads]]]
```

Argument	Default	Description
ncycles	100	Number of allocation/deallocation cycles
max_nactive	4	Max concurrent live allocations per cycle (max 24)
nthreads	1	OpenMP threads (clamped to omp_get_max_threads)

Environment Variables

Variable	Default	Description
CHECK	0	Non-zero: memset each allocation (validates writeable)

Compile-Time Knobs

Macro	Default	Description
MAX_MALLOC_MB	100	Upper bound on individual allocation size in MB
MAX_MALLOC_N	24	Array size for random-number table and alloc slots

```
## default: 100 cycles, up to 4 active allocations, 1 thread
./scratch.x

## heavy load: 500 cycles, up to 12 active allocations, 4 threads
./scratch.x 500 12 4

## validate pages are writable
CHECK=1 ./scratch.x 43 8 4
```

Usage (Fortran)

```
./scratchf.x [mbytes [nrepeat]]
```

Argument	Default	Description
mbytes	100	Allocation size in MB
nrepeat	20	Number of repetitions

Single-threaded benchmark comparing `libxs_malloc` / `libxs_free` against Fortran ALLOCATE / DEALLOCATE. Reports per-iteration times (ms) and an average speedup ratio.

What Is Measured

Each cycle draws a random number of allocations (1 to max_nactive) with random sizes (1 to MAX_MALLOC_MB MB each). The cycle allocates all buffers, optionally touches them (CHECK), then frees them.

Both paths use the same randomized sequence. An untimed warm-up cycle runs before the measured loops. Reported metrics:

- calls/s (kHz) — allocation+free throughput for each allocator
- Scratch size — high-water mark of the pool (MB)
- Malloc size — peak aggregate allocation per cycle (MB)
- Scratch Speedup — ratio of pool throughput to stdlib throughput
- Fair — size-adjusted speedup: (malloc_size / scratch_size) * speedup

Setdiff

Runs small deterministic experiments for `libxs_setdiff` and `libxs_setdiff_min`. The sample writes CSV files that are useful for paper figures and for quick local proxy runs.

Build

```
cd samples/setdiff
make
```

Usage

```
./setdiff.x [outdir [sizes [reps [land_n [land_steps]]]]]
```

Positional	Default	Description
outdir	`results/setdiff`	Directory for CSV output
sizes	`128 1024 8192 65536`	Vector sizes for tolerance and scaling runs
reps	`10`	Repetitions per scaling size
land_n	`32`	Tolerance-landscape vector length
land_steps	`40`	Number of tolerance grid steps

Output

- `summary.csv` — order-independence and duplicate-consumption cases
- `landscape.csv` — sampled tolerance landscape
- `tolerance.csv` — `libxs_setdiff_min` compared with bisection
- `scaling.csv` — fixed-tolerance and automatic-tolerance timings
- `complex.csv` — complex-valued validation case

The sample uses a fixed pseudo-random seed and writes into a deterministic output directory by default.

Shuffle

Benchmarks three shuffling strategies and compares their throughput:

Label	Method
RNG-shuffle	Fisher-Yates via `libxs_rng_u32` (reference baseline)
DS1-shuffle	`libxs_shuffle` — deterministic in-place (coprime stride)
DS2-shuffle	`libxs_shuffle2` — deterministic out-of-place (src to dst)

Each iteration works on an array of unsigned integers initialized to 0..n-1. On the final iteration the shuffled result can optionally be written as an MHD image file or analyzed for randomness quality.

Build

```
cd samples/shuffle
make
```

Usage

```
./shuffle.x [nelems [elemsize [niters [repeat]]]]
```

Positional	Default	Description
nelems	64 MB / elemsize	Number of elements
elemsize	4	Element size in bytes (1, 2, 4, or 8)
niters	1	Shuffle passes per timed measurement
repeat	3	Timed iterations (first is warmup)

Environment Variables

Variable	Default	Description
RANDOM	0	Non-zero: count inversions and report randomness (rand=%)
SPLIT	1	Partitioning depth for the STATS metrics
STATS	0	Non-zero: print partition imbalance (imb) and distance (dst)

```
## Default run (64 MB of 4-byte elements, 3 repeats)
./shuffle.x

## 1M elements of 8 bytes, 4 shuffle passes, 5 repeats
./shuffle.x 1000000 8 4 5

## Enable randomness quality metric
RANDOM=1 ./shuffle.x 10000 4 1 3

## Enable partition statistics with split depth 2
STATS=1 SPLIT=2 ./shuffle.x
```

What Is Measured

Reported bandwidth is 2 * data-size / time (in MB/s). The factor of two accounts for reading and writing each element during the shuffle. The first iteration is excluded from the average.

Optional quality metrics:

- rand — Enabled by RANDOM=1. Inversions are counted via merge-sort and compared against the expected count for a random permutation (n*(n-1)/4) to give a percentage.
- dst — Manhattan distance of the element sum from the expected uniform value, split into hierarchical partitions (SPLIT).
- imb — Partition imbalance, measuring how unevenly element sums are distributed across sub-partitions (SPLIT).

MHD Image Output When RANDOM is not set and the element size is 1, 2, 4, or 8 bytes, the final shuffled array is written as an MHD image per method (shuffle_rng.mhd, shuffle_ds1.mhd, shuffle_ds2.mhd). The images are shaped close to square (isqrt(n) x n/isqrt(n)) and converted to unsigned 8-bit via modulus.

Compile-Time Knobs

Variable	Default	Description
OMP	0	Enable OpenMP (not used by this sample)
SYM	1	Include debug symbols (-g)
BUBBLE_SORT	undefined	Define (-DBUBBLE_SORT) for O(n^2) inversion counter

Spatial

Samples for spatial data structures and queries built on libxs primitives: space-filling curves (Hilbert, Morton), kd-tree partition and search, and spatial pair counting. Each topic uses a filename prefix to group related files in this flat directory.

Prefix	Topic
`stratify_*`	3D-to-2D layout via space-filling curves
`paircnt_*`	Spatial pair counting (planned)

Underlying libxs primitives (from `libxs_perm.h`):

- Hilbert and Morton curve encode/decode (N-D stratification)
- kd-tree build, partition, nearest-neighbor search
- Sort by spatial locality (various methods)

Building

Build libxs first so that `lib/libxs.so` exists:

```
cd ../..
make PEDANTIC=2
cd samples/spatial
make
```

Optional Python adapters need `numpy` and `h5py`:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
```

Stratification (stratify_*)

Folding higher-dimensional data into lower-dimensional layouts using space-filling-curve stratification. The mapping can be computed once for a fixed detector or simulation grid and reused by any downstream pipeline.

The transform encodes each source voxel coordinate as a 3D Hilbert or Morton key and decodes it as a 2D coordinate:

```
3D coordinate -> 3D curve rank -> inverse 2D curve coordinate
```

This preserves the deterministic curve order while giving downstream code a 2D tensor layout that can be consumed by optimized 2D convolution kernels.

The samples distinguish the curve order from the 2D frame. The default `compact` frame streams voxels in curve-rank order into a dense near-square sheet, avoiding unused cells when an exact factor pair is available. The `canonical` frame decodes the finite 3D curve rank through the corresponding 2D curve, preserving the canonical destination curve coordinate at the cost of possible empty cells.

Usage The default target runs the dense 3D sample:

```
make dense3d
make dense3d FRAME=canonical
```

The older `make volume` spelling remains as a compatibility alias.

Direct invocation:

```
python3 stratify_dense3d.py --shape 8 16 16 --curve hilbert \
  --frame compact --out sheet.pgm
python3 stratify_dense3d.py --shape 8 16 16 --curve morton \
  --frame canonical --map-csv map.csv
```

HDF5 input is optional and requires `h5py` and `numpy`. The input dataset may be a single `D,H,W` volume, a batched `N,D,H,W` dataset, a channelled `N,C,D,H,W` or `N,D,H,W,C` dataset, or a flattened `N,V` dataset when `--hdf5-reshape D H W` is supplied. The default `auto` layout recognizes the 3DGAN Caffe sample layout `N,C,D,H,W`:

```
python3 stratify_dense3d.py --hdf5 /tmp/3Dgan-risk-audit/caffe/train.h5 \
  --hdf5-dataset ECAL --hdf5-event 0 --hdf5-channel 0 \
  --curve hilbert --frame compact --out sheet.pgm --out-hdf5 sheet.h5
```

The Makefile exposes the same path without making the default target depend on external data:

```
make hdf5 HDF5_FILE=/tmp/3Dgan-risk-audit/caffe/train.h5 FRAME=compact
```

For flattened HDF5 files, pass the dataset name, layout, and target 3D shape:

```
make hdf5 HDF5_FILE=dataset_2_1.hdf5 HDF5_DATASET=showers \
  HDF5_LAYOUT=flat HDF5_RESHAPE="45 16 9"
```

Public calorimeter HDF5 datasets with a documented download path are available from the CaloChallenge project. The challenge HDF5 files contain `incident_energies` and flattened `showers` datasets, so use `--hdf5-dataset showers` and `--hdf5-reshape D H W` with the geometry stated for the selected dataset. Useful starting points are:

Source	Format / notes
CaloChallenge	Overview, geometry, evaluation.
Dataset 1	ATLAS photons/pions, flat showers.
Dataset 2	Electrons, reshape to `45 16 9`.
Dataset 3	Hi-gran electrons, `45 50 18`.
Legacy sets	Submitted outputs for repro figures.

Arguments:

Option	Description
`--shape D H W`	Volume shape (default `8 16 16`).
`--curve`	`hilbert` or `morton`.

Option	Description
`--frame`	`compact` or `canonical`.
`--libxs`	Path to libxs shared library.
`--hdf5`	Read volume from an HDF5 file.
`--hdf5-dataset`	HDF5 dataset name (default `ECAL`).
`--hdf5-layout`	`auto`/`dhw`/`ndhw`/`ncdhw`/`flat`.
`--hdf5-reshape`	Reshape flat data to `D H W`.
`--hdf5-event`	Event index (default `0`).
`--hdf5-channel`	Channel index (default `0`).
`--out`	Write 8-bit PGM of the sheet.
`--map-csv`	Write `src,z,y,x,dst,v,u` rows.
`--out-hdf5`	Write sheet+map to HDF5 file.

MedMNIST3D The `stratify_medmnist3d.py` sample reads one standardized MedMNIST3D volume from an `.npz` file and applies the same 3D-to-2D stratification primitive. It is a dataset adapter rather than a training script, which keeps the dependency surface small: only `numpy` is required to parse the MedMNIST file. The sample does not require PyTorch or the `medmnist` Python package unless you use those tools separately to download the data.

The official MedMNIST distribution is available from the project page and Zenodo: MedMNIST and Zenodo DOI. The 3D subsets are `organmnist3d`, `nodulemnist3d`, `adrenalmnist3d`, `fracturemnist3d`, `vesselmnist3d`, and `synapsemnist3d`.

Direct invocation with an explicit NPZ file:

```
python3 stratify_medmnist3d.py --npz ~/.medmnist/organmnist3d.npz \
  --split train --index 0 --curve hilbert --frame compact \
  --out stratified_medmnist3d.pgm \
  --map-csv stratified_medmnist3d.csv \
  --label-csv stratified_medmnist3d_label.csv
```

The Makefile exposes the same path without making the default target depend on external data:

```
make medmnist3d MEDMNIST3D_NPZ=~/.medmnist/organmnist3d.npz FRAME=compact
```

The same NPZ input can be used with the metrics script to report invariants, locality distortion, and LIBXS Foeppl fingerprint distances for a selected volume:

```
make medmnist3d-metrics MEDMNIST3D_NPZ=~/.medmnist/organmnist3d.npz \
  MEDMNIST3D_SPLIT=test MEDMNIST3D_INDEX=0 FRAME=canonical
```

If `MEDMNIST3D_NPZ` is omitted, the script looks for a dataset under `MEDMNIST3D_ROOT` using `MEDMNIST3D_FLAG` and `MEDMNIST3D_SIZE`:

```
make medmnist3d MEDMNIST3D_ROOT=~/.medmnist MEDMNIST3D_FLAG=nodulemnist3d
```

Metrics The `stratify_dense3d_metrics.py` script reports invariants and locality distortion for the same synthetic and HDF5 inputs:

```
make metrics
python3 stratify_dense3d_metrics.py --hdf5 /tmp/3Dgan-risk-audit/caffe/train.h5 \
  --hdf5-dataset ECAL --curve hilbert
python3 stratify_dense3d_metrics.py --hdf5 dataset_2_1.hdf5 --hdf5-dataset showers \
  --hdf5-layout flat --hdf5-reshape 45 16 9 --curve morton
```

The invariant metrics check that stratification is a lossless layout transform: the total energy, reconstructed voxel values, and per-axis energy profiles match after applying the inverse map. The distortion metrics measure what changes for a convolutional model: how far source 3D grid neighbors move apart in the 2D sheet, and how often adjacent sheet cells correspond to adjacent source voxels.

The script also reports LIBXS Foeppl fingerprint metrics. These are compact multi-order L2 descriptors of value and finite-difference structure. The `fprint.source_reconstructed.diff` value should be zero for a correct lossless round trip, while `fprint.source_sheet.diff` summarizes how different the stratified 2D sheet looks as a structured field.

Geometry The current sample treats the source as a regular `D,H,W` index grid. This is enough for dense 3DGAN-style arrays and for CaloChallenge datasets after their flattened `showers` arrays are reshaped with the documented dimensions.

Supporting other detector geometries should be data-driven rather than encoded as a list of known experiments. A general geometry adapter needs one of the following descriptions:

Geometry input	Role
Integer logical coords	Direct Hilbert/Morton input.
Floating physical coords	Quantized before stratification.
Adjacency edges (optional)	Neighbor-distortion metrics.
Voxel weights (optional)	Physics metrics for unequal bins.

Pair Counting (paircnt_*) — planned

Spatial pair counting inspired by Corrfunc: given a point catalog, bin all pairs by separation distance to compute correlation functions. The approach uses libxs kd-tree partition for cell decomposition and cell-pair pruning (minimum-separation skip), with vectorized inner loops for the actual distance computation and bin assignment.

Design goals:

- Leverage `libxs_kdtree_build` / `libxs_kdtree_partition` for spatial decomposition rather than a separate grid structure.
- Cell-pair pruning: skip cell pairs whose minimum separation exceeds the largest bin edge.
- Vectorized pair-counting kernel for the inner loop (SIMD where available).
- Periodic and non-periodic boundary conditions.
- Output: binned pair counts DD(r), optionally normalized to xi(r) via Landy-Szalay with a random catalog.

Synchronization Primitives

Two binaries over `libxs_sync.h`: `sync_lock.x` for the lock kinds and `sync_barrier.x` for `libxs_barrier_t`.

sync_lock.x

Micro-benchmark for the lock implementations provided by LIBXS (`libxs_sync.h`). Measures single-thread latency (uncontended acquire/release) and multi-thread throughput (mixed read/write workload) for every compiled lock kind:

Lock kind	Description
LIBXS_LOCK_DEFAULT	Compile-time default (typically atomic)
LIBXS_LOCK_SPINLOCK	OS-native or CAS-based spin lock (if available)
LIBXS_LOCK_MUTEX	OS-native mutex / pthread_mutex_t (if available)
LIBXS_LOCK_RWLOCK	Reader/writer lock / pthread_rwlock_t

The default lock is always benchmarked. The remaining kinds are conditionally compiled depending on platform support.

Build

```
cd samples/sync
make
```

OpenMP is enabled by default (OMP=1) for multi-threaded tests.

Run

```
./sync_lock.x [nthreads] [wratio%] [work_r] [work_w] [nlat] [ntpt]
```

Argument	Default	Description
nthreads	all available	Number of OpenMP threads
wratio%	5	Percentage of write operations (0-100)
work_r	100	Simulated work inside read-critical section (cy)
work_w	10 * work_r	Simulated work inside write-critical section
nlat	2000000	Iterations for latency measurement
ntpt	10000	Iterations per thread for throughput measurement

```
./sync_lock.x 4 5 100 1000

LIBXS: default lock-kind "atomic" (Other)
LIBXS: pauses before yielding 4096 (lock), 1024 (barrier)

Latency and throughput of "atomic" (default) for nthreads=4 wratio=5% ...
        atomic ro-latency: 11 ns (call/s 91 MHz, 33 cycles)
        atomic rw-latency: 11 ns (call/s 90 MHz, 33 cycles)
        atomic throughput: 0 us (call/s 9128 kHz, 328 cycles)
```

Each measured line names the lock kind it belongs to, and the run reports the pause budgets it was built with, because a contended figure depends on those more than on anything else on the command line.

To reach the regime a hot internal lock lives in - a critical section of a few instructions, taken by every thread - ask for a short one and nothing but writes:

```
./sync_lock.x 384 100 1 1 100000 5000
```

The defaults instead describe a long critical section, where the lock is held long enough that waiting for it is not the cost.

Measurement Details

- RO-latency: uncontended read-lock acquire/release pairs (4x unrolled), reported as nanoseconds per operation and TSC cycles.
- RW-latency: uncontended write-lock acquire/release pairs (4x unrolled).
- Throughput: all threads run a mixed read/write workload governed by wratio%. Simulated work inside the critical section is subtracted so only synchronization overhead is reported.

sync_barrier.x

Cost of a rendezvous over `libxs_barrier_t`, and of the broadcast that hands one task's value to the rest.

```
./sync_barrier.x [nthreads] [nrepeat]
```

Argument	Default	Description
nthreads	all available	Tasks in the team
nrepeat	100000	Rendezvous per measurement

The team is asked how large it actually is rather than told: a barrier initialized for more tasks than the runtime grants waits for one that never arrives.

Both are checked and the binary fails if either is wrong, because a rendezvous that releases a task early is fast and worthless: every task stamps a slot of its own and reads all of them after the rendezvous, and the broadcast publishes a value that changes each round.

The stamp check runs in a pass of its own, outside the timing. Reading every stamp is work proportional to the team, so at a wide one it costs more than the rendezvous it checks, and timing the two together would report the check as if it were the barrier.

Expect a flat barrier to cost more once the team reaches the number of cores: every waiting task spins, so a team that over-subscribes the machine spends its time taking cycles away from the task it is waiting for. That is a property of the primitive, not of the measurement, and it is the reason to keep a rendezvous out of a tight loop.

SYRK / SYR2K Sample

Demonstrates symmetric rank-k and rank-2k updates using the LIBXS dispatch-and-call model. The sample validates correctness against a plain Fortran reference and reports performance.

Operations

SYRK: C := alpha * A * A^T + beta * C (lower triangle) SYR2K: C := alpha * (A * B^T + B * A^T) + beta * C (upper triangle)

Only the specified triangle of C is written; the other triangle is left untouched.

Build

```
make
```

Requires a Fortran compiler (gfortran, ifort, or ifx). The Makefile picks up the compiler from the top-level Makefile.inc. MKL or BLAS linkage is enabled (BLAS=1) so that dispatch can use JIT-compiled kernels when available.

Run

```
./syrk.x [N [K [nrepeat [direct]]]]
```

Arguments (all optional, positional):

```
N         Matrix dimension of C (N x N).  Default: 64
K         Inner dimension (columns of A).  Default: N
nrepeat   Number of timed repetitions.     Default: 100
direct    Interface selection.             Default: 0
            0 = config (dispatch + call separately)
            1 = direct (one-shot generic)
```

Example Output

```
syrk(F): N=64 K=64 nrepeat=100

--- libxs_syrk (lower) ---
  max error (lower): 0.00000E+00

--- libxs_syr2k (upper) ---
  max error (upper): 0.00000E+00

--- SYRK performance ---
  time:      0.002 s (100 calls)
  perf:      28.4 GFLOPS/s
```

Notes

- The dispatch step (libxs_syrk_dispatch / libxs_syr2k_dispatch) returns a pointer to a registry-owned config. This pointer remains valid until libxs_finalize or the registry is destroyed. There is no need to release it manually.
- Internally, SYRK/SYR2K decompose into GEMM tiles on the diagonal and off-diagonal blocks. The dispatched GEMM kernel (MKL JIT, LIBXSMM, or fallback BLAS) handles the inner loop.
- Scratch memory for the temporary full-panel product is managed via a thread-local buffer that grows on demand and is freed at finalization.

Metatoken Sample

The sample encodes the same input with each `libxs_tokenizer_t` granularity, prints the resulting logical metatokens, and verifies exact decoding.

Build

```
cd ../..
make PEDANTIC=2
make -C samples/tokenizer PEDANTIC=2
```

Usage

```
./samples/tokenizer/tokenizer.x
./samples/tokenizer/tokenizer.x "token tokenization tokenizer token"
printf '%s\n' 'UTF-8 text here' | ./samples/tokenizer/tokenizer.x -
```

Output reports the input byte count followed by each token's kind, logical byte length, physical cell count, and sentence-boundary flag. A nonzero exit status means encoding or round-trip verification failed.